UNIT: 1 Introduction to Software Testing

1.1 Software testing and its importance

Definition:

Software testing is a technique used to identify errors, defects, or missing requirements in software and ensure that the product meets customer expectations.

Objectives of Software Testing

1. Find defects before software reaches users.
2. Verify that software meets requirements.
3. Validate that the product fulfills user needs.
4. Improve software quality.
5. Reduce maintenance costs.
6. Increase customer satisfaction.

Why is Software Testing Important?

1. Ensures Quality: Testing ensures the software functions correctly and provides a good user experience.
2. Reduces Cost: Fixing bugs during development is much cheaper than fixing them after deployment.
3. Enhances Security: Testing identifies vulnerabilities and security loopholes.
4. Improves Reliability: Reliable software performs consistently under different conditions.
5. Customer Satisfaction: A bug-free application increases user confidence and satisfaction.

Example

Consider an online banking application:

Without Testing:

- Incorrect money transfer
- Wrong account balance
- Security issues

With Testing:

- Accurate transactions
- Secure login
- Reliable performance

Advantages of Software Testing

- Improves product quality
- Increases customer trust
- Reduces maintenance costs

VPMP POLYTECHNIC

- Ensures compliance with requirements
- Detects defects early

1.2 Software Testing Terminologies.

1. Error

An error is a human mistake made during software development.

Example:

A programmer writes:

if(a=b)

instead of

if(a==b)

This mistake is called an **Error**.

2. Defect (Bug)

A defect is a flaw in software caused by an error.

Example:

The login page crashes when an incorrect password is entered.

3. Failure

A failure occurs when software does not perform its intended function.

Example:

The ATM deducts money but does not dispense cash.

4. Test Case

A test case is a set of conditions used to verify a specific functionality.

VPMP POLYTECHNIC

Example:

Test Case ID	**TC01**
Objective	Verify Login
Input	Valid Username & Password
Expected Result	Login Successful

5. Test Suite

A collection of test cases grouped together.

Example:

- Login Test Cases
- Registration Test Cases
- Payment Test Cases

Together they form a Test Suite.

6. Test Scenario

A high-level description of what needs to be tested.

Example:
"Verify User Login Functionality"

7. Test Data

Input values used during testing.

Example:

Username: admin
Password: admin123

8. Test Environment

Hardware, software, network, and database setup where testing is performed.

9. Test Plan

VPMP POLYTECHNIC

A document describing:

- Scope
- Objectives
- Resources
- Schedule
- Testing strategy

10. Severity

Severity indicates the impact of a defect on the system.

Levels:

- Critical
- Major
- Minor
- Trivial

11. Priority

Priority determines how quickly a defect should be fixed.

Levels:

- High
- Medium
- Low

Difference between Severity and Priority

Severity	**Priority**
Impact on system	Urgency to fix
Technical concern	Business concern

1.3 Verification and Validation (V Model), Quality Assurance, Quality Control.

Verification

Definition

Verification checks whether the product is being built correctly according to specifications.

VPMP POLYTECHNIC

Question:
"Are we building the product right?"

Activities

- Reviews
- Walkthroughs
- Inspections
- Requirement analysis

Characteristics

- Static process
- No code execution required
- Performed during development

Validation

Definition

Validation checks whether the correct product has been built.

Question:
"Are we building the right product?"

Activities

- Software testing
- User acceptance testing
- System testing

Characteristics

- Dynamic process
- Code execution required

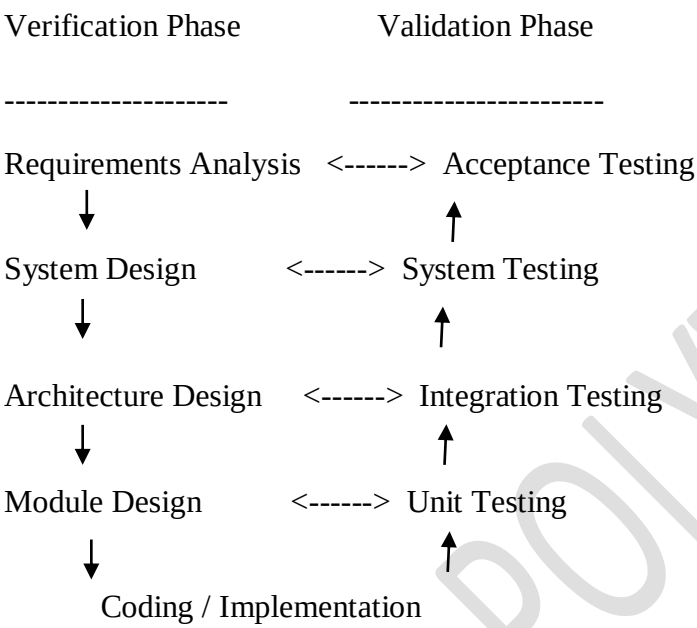
Difference between Verification and Validation

Verification	Validation
Checks specifications	Checks user needs
Static activity	Dynamic activity
No execution	Requires execution
Prevents defects	Detects defects

VPMP POLYTECHNIC

V-Model (Verification and Validation Model)

- The V-Model is an extension of the Waterfall Model where testing activities are planned alongside development activities.
- The process is represented as a "V" shape, where the left side contains development and specification activities, the bottom contains coding, and the right side contains corresponding testing activities.
- The V-Model is based on the principle that:
  - **Verification**: "Are we building the product right?"
  - **Validation**: "Are we building the right product?"



Advantages

- Early defect detection
- Better quality
- Clear testing phases

Disadvantages

- Less flexible
- Costly if requirements change

VPMP POLYTECHNIC

Quality Assurance (QA)

Definition: QA is a process-oriented activity focused on preventing defects.

Goal: Improve development processes.

Examples

- Process audits
- Standards compliance
- Training

Characteristics

- Preventive
- Process-focused

Quality Control (QC)

Definition: QC is a product-oriented activity focused on identifying defects.

Examples

- Testing
- Inspection
- Reviews

Characteristics

- Corrective
- Product-focused

QA vs QC

QA	QC
Process-oriented	Product-oriented
Prevents defects	Finds defects
Proactive	Reactive
Verification	Validation

VPMP POLYTECHNIC

1.4 Software development life cycle (SDLC) and testing phases

The Software Development Life Cycle (SDLC) is a structured process used to plan, design, develop, test, deploy, and maintain software. It ensures a systematic workflow and helps align software development with business goals and user requirements.

- Provides a clear and organized framework for managing development phases
- Helps in early detection of defects, reducing overall cost and time
- Ensures high-quality software delivery that meets user expectations

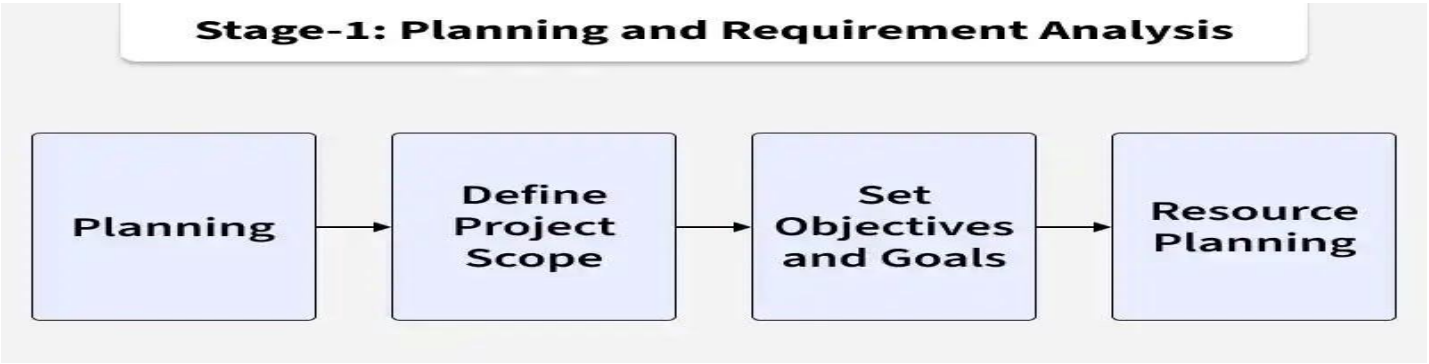


Stages of the Software Development Life Cycle
The Software Development Life Cycle (SDLC) typically consists of six or seven stages, depending on the development model used.

Stage 1: Planning & Feasibility Analysis

This stage determines whether the project is technically, financially, and operationally feasible.

- **Activities:** Feasibility analysis, cost estimation, scheduling, resource planning
- **Output:** Project Plan, Feasibility Report
- **Key Roles:** Project Managers, Senior Engineers, Stakeholders

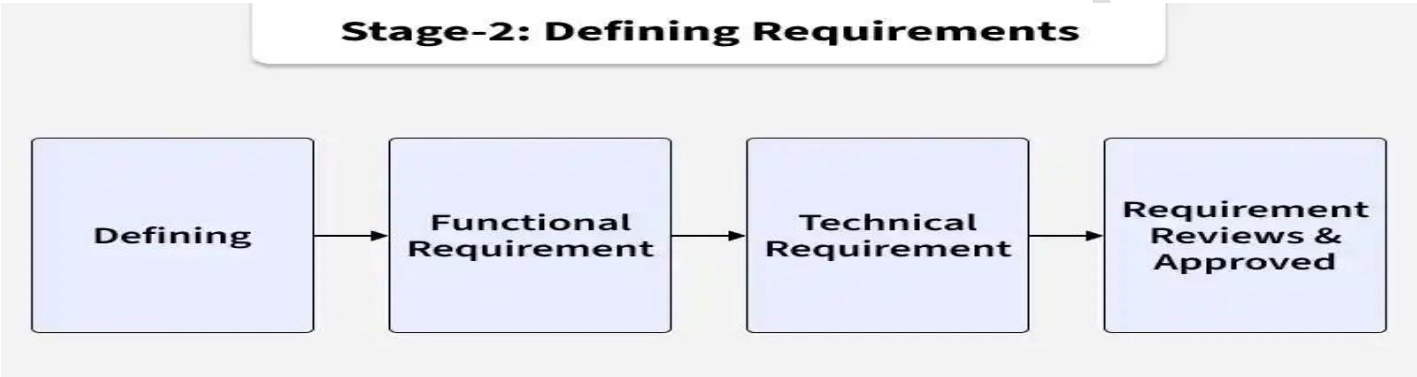


stage-1

Stage 2: Requirement Specification (SRS)

In this stage, detailed functional and non-functional requirements are documented clearly and approved by stakeholders.

- **Activities:** Requirement gathering, validation, documentation
- **Output:** Software Requirement Specification (SRS)
- **Key Roles:** Business Analysts, Product Owners

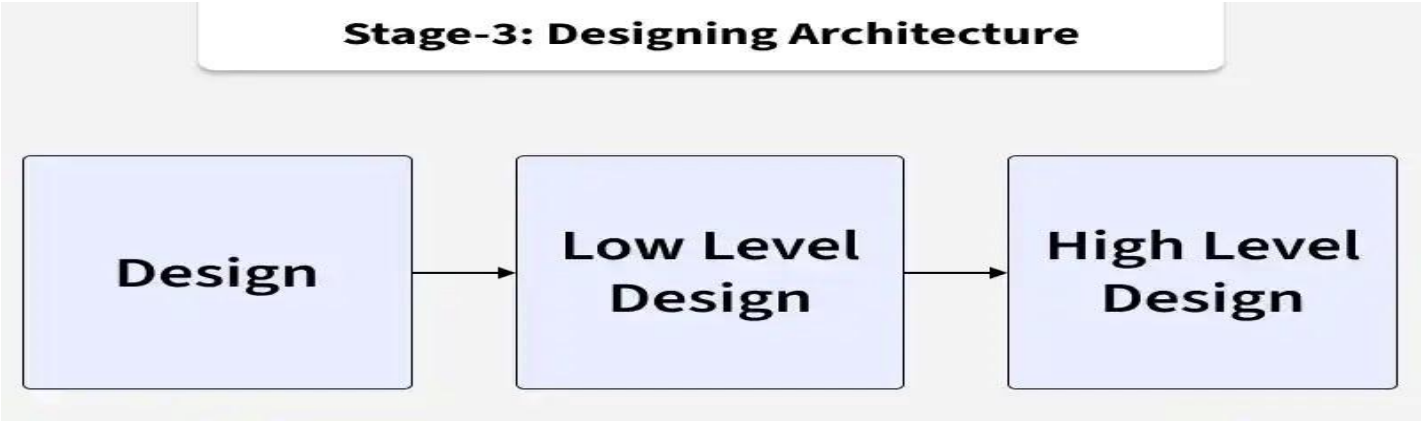


Stage-2

Stage 3: System Design

In this stage, the approved requirements are transformed into a technical blueprint for implementation.

- **High-Level Design (HLD): Defines system architecture, technology stack, database design, and major modules.**
- **Low-Level Design (LLD):** Specifies component logic, APIs, data structures, and workflows.
- **Output:** Design Document Specification (DDS)

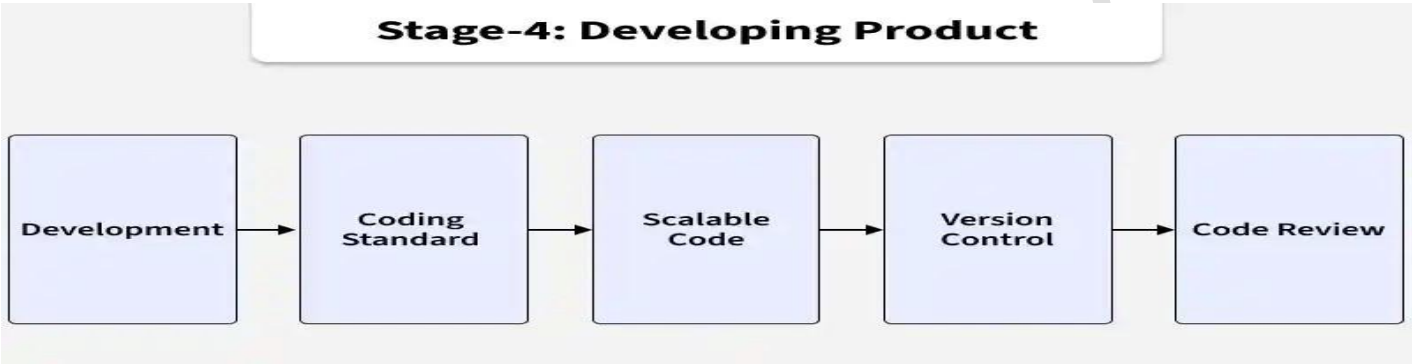


Stage-3

Stage 4: Development (Coding)

Developers build the software based on the approved design.

- **Activities:** Coding, code reviews, unit testing, version control management
- **Tools:** IDEs, version control systems, debuggers
- **Output:** Source code, executable application
- **Key Roles:** Frontend, Backend, Full Stack Developers



Stage-4

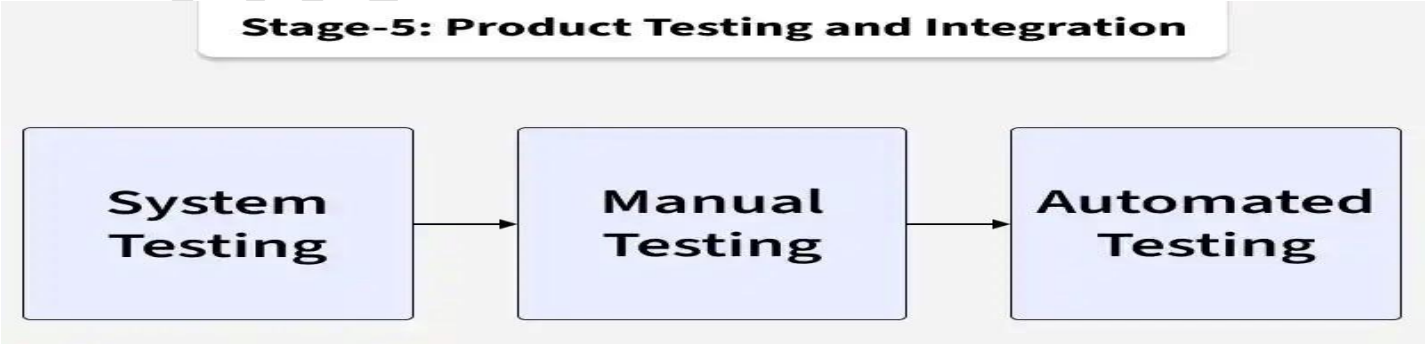
Stage 5: Testing

Testing ensures the software meets requirements and is free from defects before release.

Types of Testing include:

- **Unit Testing**: Verifies individual components
- **Integration Testing**: Ensures modules work together
- **System Testing:** Validates the complete system
- **User Acceptance Testing (UAT)**: Confirms business requirements are met

Output: Test cases, defect reports, and quality metrics.

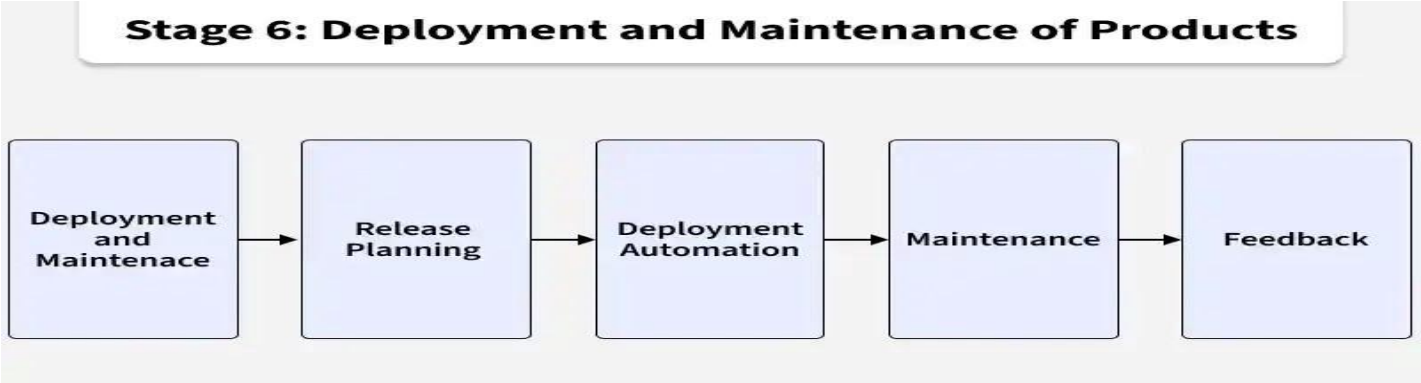


Stage-5

Stage 6: Deployment

The tested software is released to users.

- **Activities:** Production setup, deployment, smoke testing
- **Modern Approach:** Continuous Integration and Continuous Deployment (CI/CD) pipelines for faster and reliable releases
- **Output:** Live application
- **Key Roles:** DevOps Engineers, Release Managers



Stage-6

Stage 7: Maintenance

Post-deployment support ensures long-term usability.

- **Activities:** Bug fixes, performance tuning, updates, feature enhancements
- **Output:** Patches, updates, new versions
- **Key Roles:** Support Engineers, Developers

1.5 STLC – Software Testing Life Cycle in detail, SDLC

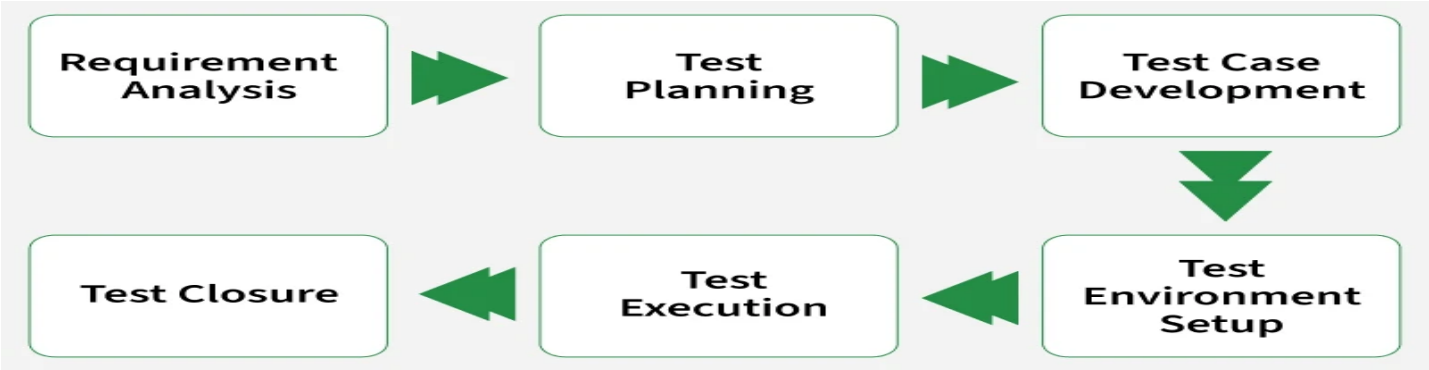
vs STLC

The Software Testing Life Cycle (STLC) is a structured process that defines the steps involved in testing a software product. It ensures that the application meets quality standards and user expectations. STLC is an essential part of the Software Development Life Cycle (SDLC) and guides QA teams in testing activities.

- Provides a systematic approach to plan, execute, and manage testing
- Helps in early detection of defects and improves software quality
- Ensures complete test coverage and proper reporting of results

Phases of STLC

There are six major phases of the Software Testing Life Cycle (STLC) that we are discussing here in detail.



Phases of STLC

1. Requirement Analysis

Requirement Analysis is the first phase where the QA/testing team understands what needs to be tested.

- Review the Software Requirements Document (SRD) and related documents
- Interview stakeholders to gather additional information
- Identify ambiguities, inconsistencies, or missing requirements
- Review non-functional requirements such as performance, security, usability, and compliance
- Identify potential risks or challenges that may impact testing

2. Test Planning

Test Planning is the most crucial phase where the overall test strategy and plan are created. The major activities carried out during the Test Planning phase include:

- Define testing objectives, scope, and priorities
- Develop a test strategy, including selection of manual or automated testing techniques
- Identify required testing environments, tools, and resources
- Define entry and exit criteria for each phase
- Estimate time, cost, and resource requirements
- Identify test deliverables and key milestones
- Assign roles and responsibilities to the testing team
- Incorporate risk-based testing to prioritize critical functionalities
- Review and approve the test plan

3. Test Case Development

The Test Case Development phase testers design detailed test cases and prepare the necessary test data. The main activities performed during the Test Case Development phase in include:

- Identifying the test cases that will be developed
- Writing test cases that are clear, concise and easy to understand
- Creating test data and test scenarios that will be used in the test cases
- Identifying the expected results for each test case
- Reviewing and validating the test cases
- Updating the requirement traceability matrix (RTM) to map requirements to test cases

4. Test Environment Setup

Test Environment Setup defines the hardware, software and network conditions under which testing will be executed. The activities involved in the Test Environment Setup phase include:

- Install and configure required software, tools and databases.
- Set up servers, browsers, operating systems and devices.
- Prepare access credentials and permissions.
- Validate the environment before test execution.
- Test environments can be physical, virtual, or cloud-based. For Agile/DevOps environments, ensure integration with CI/CD pipelines and test automation tools.

5. Test Execution

In Test Execution phase prepared test cases are executed in the defined environment. The activities performed during the Test Execution phase include:

- Run manual or automated test cases.
- Log defects with details like severity and priority.
- Retest fixed defects (defect retesting).
- Perform regression testing if required.
- Collect and analyze test results.
- Document and share test reports.

6. Test Closure

Test Closure is the final phase where testing activities are completed and documented. The final activities carried out during the Test Closure phase include:

- Prepare a Test Summary Report (test cases executed, pass/fail count, defects found/resolved).
- Ensure all defects are tracked and closed.
- Clean up the test environment.
- Archive test cases, data and reports.
- Conduct a retrospective for lessons learned.
- Share knowledge with stakeholders.

STLC vs SDLC

Aspect	SDLC (Software Development Life Cycle)	STLC (Software Testing Life Cycle)
Definition	A process that defines all phases of software development, from requirements gathering to deployment and maintenance.	A process that defines all phases of software testing, from requirement analysis to test closure.
Focus	Focuses on building the software.	Focuses on verifying and validating the software.

Phases	**Requirement gathering, Design, Development, Testing, Deployment, Maintenance.**	**Requirement analysis, Test planning, Test case development, Test environment setup, Test execution, Test closure.**
Performed By	**Developers, business analysts, project managers, QA team (partly).**	**QA/testing team primarily.**
Deliverables	**Software product, design documents, user manuals, deployment package.**	**Test plan, test cases, defect reports, test summary, closure report.**
Objective	**To deliver a working software product that meets user requirements.**	**To ensure the product is defect-free and high quality before release.**
Relation	**Covers the entire lifecycle of the software.**	**Part of SDLC, focused only on testing.**

SDLC vs STLC

SDLC	STLC
Develops software	Tests software
Focuses on creation	Focuses on quality
Starts from requirements	Starts after requirement analysis
Ends with maintenance	Ends with closure report

1.6 Types of testing (manual vs. automated)

Software Testing is a critical phase of the Software Development Life Cycle (SDLC) that ensures a software application works correctly and meets user requirements. It helps identify defects and improve the overall quality, reliability, and performance of the software.

- Ensures the application functions as expected and meets user needs.
- Helps detect bugs early and improves software quality.
- Enhances reliability, security, and performance of the system.

Types Based on Execution Method

Software testing can be classified based on how test cases are executed, either manually by testers or automatically using testing tools and scripts.

Manual Testing

Manual Testing is a software testing technique in which test cases are executed manually by testers without using automation tools.

- **Accurate visual feedback:** Helps detect UI/UX issues like layout, design, and text changes effectively.
- **Cost-effective:** Does not require expensive tools or advanced technical skills.
- **No coding required:** Suitable for beginners as it does not require programming knowledge.
- **Flexible for changes:** Easily adapts to frequent or unplanned changes in the application.

Automation Testing

Automation Testing is a technique where test cases are executed using scripts and automation tools. It is mainly used for repetitive and time-consuming tasks, improving efficiency, accuracy, and test coverage.

- **Faster execution:** Runs tests automatically, saving time and effort.
- **Improved reliability:** Ensures consistent and accurate results without human errors.
- **Higher test coverage:** Allows execution of large and complex test cases.
- **Reduced human effort:** Minimizes manual work and post-release defects.

Types Based on Testing Technique

Software testing can be categorized based on the testing technique used.

White Box Testing

White Box Testing is a software testing technique in which the internal structure, logic, and source code of an application are examined. It is typically performed by developers to verify that the code functions correctly and efficiently.

- Focuses on internal code, logic, and program structure.
- Requires knowledge of programming and code implementation.
- Helps identify errors in loops, conditions, paths, and data flow.

Black Box Testing

Black-Box Testing is a software testing technique where the functionality of the application is tested without knowing the internal code or structure. It focuses on inputs and expected outputs to verify whether the system behaves correctly.

- Focuses on functionality rather than internal code.
- Does not require programming knowledge.
- Tests application using inputs and expected outputs.

Gray Box Testing

Gray Box Testing is a software testing technique that combines both white box and black box testing approaches. The tester has partial knowledge of the internal structure but tests the application from a user's perspective.

- Combines both functional and structural testing.
- Requires partial knowledge of the system's internal code.
- Helps identify defects related to data flow and integration.

Types Based on Testing Purpose

Software testing can also be classified based on the purpose of testing, such as validating functionality, performance, security, reliability, and overall system behavior.

Functional Testing

Functional Testing verifies whether the software functions according to the specified business requirements.

Types of Functional Testing

- Unit Testing: It is Tests individual functions, methods, or modules in isolation. It helps identify defects at the earliest stage of development.
- Integration Testing: It is Verifies that different modules or components work correctly together. It ensures proper data flow and communication between integrated parts.
- System Testing: Tests the complete application as a whole against specified requirements. It validates the overall functionality of the system.
- User Acceptance Testing (UAT): Conducted by end users or clients to verify that the software meets business requirements. It is the final testing phase before deployment.
- Smoke testing: Checks the basic and critical functionalities of a new build. It determines whether the build is stable enough for further testing.
- Sanity Testing: Performs a quick validation of specific changes or bug fixes. It ensures that recent modifications work as expected.
- Interface Testing: Tests the interaction between different systems, applications, or APIs. It ensures that data is exchanged correctly across interfaces.
- Regression Testing: It Verifies that recent changes or bug fixes have not affected the existing functionality of the application.
- API Testing: Verifies that APIs function correctly by validating requests, responses, data exchange, and business logic between applications.

Non-Functional Testing

Non-Functional Testing evaluates the performance, reliability, scalability, usability, and security of the software. It focuses on how the system performs under different conditions rather than specific functionalities.

Types of Non-functional Testing

- Performance Testing: Measures the speed, responsiveness, and stability of an application under various workloads. It ensures the system meets performance requirements.
- Usability Testing: Evaluates how easy and intuitive the application is for users. It helps improve the overall user experience and satisfaction.
- Security Testing: Identifies vulnerabilities and weaknesses that could be exploited by attackers. It helps protect data and prevent unauthorized access.
- Reliability Testing: Checks whether the software can perform consistently without failures over a specified period. It ensures dependable operation under normal conditions.
- Compatibility Testing: Verifies that the application functions correctly across different browsers, devices, operating systems, and network environments.

Types of Performance Testing

- Load Testing: Evaluates the application's performance under expected user load and normal working conditions. It helps identify performance bottlenecks before deployment.
- Stress Testing: Tests the system beyond its normal operating limits to determine its breaking point. It helps assess stability and error-handling capabilities under extreme conditions.
- Spike Testing: Measures how the application responds to sudden increases or decreases in user load. It ensures the system can handle unexpected traffic spikes effectively.
- Endurance (Soak) Testing: Checks the application's performance over an extended period under a sustained workload. It helps identify memory leaks and resource utilization issues.
- Volume Testing: Evaluates the system's ability to handle large volumes of data. It ensures that performance remains acceptable as the amount of data increases.

Types of Integration Testing

- Big Bang Integration Testing: All modules are integrated simultaneously and tested as a complete system. While simple to implement, identifying the source of defects can be challenging.
- Top-Down Integration Testing: Testing begins with the top-level modules and progressively integrates lower-level modules. Stub programs are used to simulate modules that are not yet developed.
- Bottom-Up Integration Testing: Testing starts with the lowest-level modules and gradually moves upward. Driver programs are used to simulate higher-level modules during testing.
- Sandwich (Hybrid) Integration Testing: This approach combines both Top-Down and Bottom-Up testing methods. It allows multiple modules to be tested in parallel, reducing overall testing time.

Classification of Integration Testing

Integration Testing
- *Big Bang Integration Testing*
- *Incremental Integration Testing*
  - *Top-Down Integration Testing*
  - *Bottom-Up Integration Testing*
  - *Sandwich (Hybrid) Integration Testing*

Advantages of Software Testing
Advantages of Software Testing are:

- Improves software quality
- Detects defects early
- Reduces development cost
- Enhances security and reliability
- Improves customer satisfaction

Limitations of Software Testing
Software testing has certain limitations despite being essential for software quality assurance.

- Time-consuming process
- Increases overall development cost
- Cannot guarantee completely defect-free software

1.7 Testing principles, processes, and methodologies.

Seven Principles of Software Testing

1. Testing Shows Presence of Defects: Testing can find defects but cannot prove their absence.

2. Exhaustive Testing is Impossible: Testing every possible input is impossible.

Example: For a login system with thousands of combinations.

3. Early Testing Saves Time and Cost: Defects found early are cheaper to fix.

4. Defect Clustering: Most defects are concentrated in a small number of modules.

Also known as the **80/20 Rule (Pareto Principle)**.

5. Pesticide Paradox: Repeating the same tests eventually stops finding new defects.

Solution:Regularly update test cases.

6. Testing is Context Dependent: Testing approaches vary by application.

Example:

- Banking software
- Gaming software
- Medical software

7. Absence of Errors Fallacy: A defect-free product is useless if it doesn't meet user needs.

Software Testing Process

1. Requirement Analysis
2. Test Planning
3. Test Design
4. Environment Setup
5. Test Execution
6. Defect Reporting
7. Test Closure

Testing Methodologies

1. Black Box Testing

Focus:

- Inputs and outputs
- No knowledge of internal code

Example:
Testing login functionality.

2. White Box Testing

Focus: Internal code structure Logic paths

Example:Testing loops and conditions.

3. Grey Box Testing

VPMP POLYTECHNIC

Combination of Black Box and White Box Testing.

Tester has partial knowledge of the system.

GTU Exam Important Questions

2 Marks

1. Define Software Testing.
2. What is a Defect?
3. Define Verification and Validation.
4. What is STLC?
5. What is QA?
6. What is QC?
7. Define Test Case.
8. What is Manual Testing?

5 Marks

1. Explain V-Model with diagram.
2. Differentiate Verification and Validation.
3. Explain STLC phases.
4. Explain QA and QC.
5. Compare Manual and Automated Testing.

7 Marks

1. Explain SDLC and STLC with diagram.
2. Explain Software Testing Terminologies.
3. Discuss Seven Principles of Software Testing.
4. Explain Black Box, White Box, and Grey Box Testing.

GTU Viva Questions

- Why is testing necessary?
- What is the difference between bug and defect?
- What is RTM?
- Explain Severity and Priority.
- What is the role of QA in software projects?
- Why is exhaustive testing impossible?

VPMP POLYTECHNIC